



Estructuras de datos avanzadas - Proyecto 1

21 de agosto de 2026

Problema A. Montículo

Entrada:	standard input
Salida:	standard output
Tiempo límite:	2 seconds
Memoria límite:	256 megabytes

El equipo de sistemas de UTEC está construyendo un planificador de tareas. Cada tarea tiene una *prioridad* (mientras más pequeña, más urgente) y el planificador necesita, en todo momento, poder:

- agregar una nueva tarea con una prioridad dada,
- atender (y retirar) la tarea más urgente en cola, y
- actualizar la prioridad de una tarea ya existente para hacerla *más* urgente.

Te piden implementar exactamente esta estructura: un montículo (*heap*) de mínimos que soporte **inserción**, **extracción del mínimo** y **decremento de clave**.

Formalmente, debes simular un multiconjunto de *elementos*, cada uno identificado por un entero único (su *id*) y asociado a un valor entero (su *clave*). Inicialmente hay n elementos, con ids $1, 2, \dots, n$, donde el elemento i tiene clave a_i .

Luego debes procesar q operaciones, cada una de uno de los siguientes tres tipos:

- **1 v** — se inserta un nuevo elemento con clave v . Los ids se asignan de forma consecutiva en el orden en que se procesan las operaciones de inserción: la k -ésima operación de tipo 1 de todo el input crea el elemento con id $n + k$.
- **2** — se retira del montículo el elemento de clave mínima. Si hay más de un elemento con la clave mínima, se retira el que tiene el **id más pequeño** entre ellos. Se garantiza que en el momento de esta operación el montículo no está vacío.
- **3 id v** — el elemento con el id dado (que se garantiza sigue presente en el montículo, es decir, no ha sido retirado todavía) cambia su clave a v . Se garantiza que v es estrictamente menor que la clave actual de ese elemento (es decir, siempre es un decremento real).

Nota: Este proyecto tiene fecha límite el día 28 de agosto a las 8:00 p.m. Además, se debe presentar un **reporte de 2-3 páginas** explicando el diseño de implementación, la correctitud algorítmica y el análisis de complejidad de la estructura. **Es obligatorio implementar su propio heap (no está permitido heap binario ni algún contenedor nativo del lenguaje usado) para este problema, caso contrario no se les dará el puntaje correspondiente.**

Entrada

La primera línea contiene dos enteros n y q : la cantidad inicial de elementos y la cantidad de operaciones.

La segunda línea contiene n enteros $a_1, a_2, \dots, a_n$: la clave inicial del elemento con id i , para cada $1 \leq i \leq n$.

Cada una de las siguientes q líneas describe una operación, en uno de los tres formatos descritos arriba.

Salida

Por cada operación de tipo 2, en el orden en que aparecen, imprime en una línea el id del elemento retirado.

Límites

Este problema tiene subtareas, por lo cual es posible obtener un puntaje en base a las subtareas que logren ser resueltas.

- $1 \leq n \leq 3 \cdot 10^5$.
- $1 \leq a_i \leq 10^9$ para cada $i = 1, \dots, n$.
- $1 \leq v \leq 10^9$ para toda consulta.
- En toda operación 2, el montículo tiene al menos un elemento presente en ese momento.

Grupo 1 (3 puntos)

- $1 \leq n, q \leq 2000$

Grupo 2 (9 puntos)

- Sin restricciones adicionales.

Ejemplos

standard input	standard output
3 5 5 3 8 1 1 3 3 2 2 2 1 10	4 3
2 4 7 7 2 3 2 4 1 4 2	1 2

Nota

Nota sobre el primer ejemplo. Los elementos iniciales son $1 \rightarrow 5$, $2 \rightarrow 3$, $3 \rightarrow 8$. La operación 1 1 crea el elemento 4 con clave 1 (porque $n = 3$ y es la primera inserción). La operación 3 3 2 baja la clave del elemento 3 de 8 a 2. La primera extracción retira al elemento 4 (clave 1, la mínima). La segunda extracción retira al elemento 3 (clave 2, la mínima entre los que quedan: $1 \rightarrow 5$, $2 \rightarrow 3$, $3 \rightarrow 2$). La última operación inserta el elemento 5 (clave 10) y no produce salida.

Nota sobre el segundo ejemplo. Ambos elementos iniciales tienen clave 7; al empatar, se retira el de menor id (el 1). Luego el elemento 2 baja a clave 4 y se inserta el elemento 3 también con clave 4: vuelven a empatar, y se retira el de menor id entre ambos (el 2).